High Performance Computing methods usage in parallelization of a car following model traffic simulation

Antoni Łażewski (I6399624), Maja Rzeszotarska (I6445332), Lucian Uritu (I6380834), David Jimenez Lorenzo (I6384857), Ivars Svārpstons (I6405943), Lars Vandingenen (I6402052)

Maastricht University, Department of Advanced Computing Sciences
Project 2-2, Group HPC-03
June 9, 2026

Abstract :

In this paper, we investigate the effectiveness of High-Performance Computing methods for accelerating vehicle-level traffic simulations in SUMO (Simulation of Urban MObility). We present a performance evaluation on a real-world network of Maastricht, the Netherlands, comparing parallel Monte Carlo execution, TraCI-based multi-instance control, and MPI-based spatial partitioning. We evaluate performance in terms of speedup, efficiency, accuracy, and load balancing across multiple core counts and different partitioning algorithms. Our results show that Monte Carlo execution scales best, reaching a maximum speedup of $14.9x$ at 32 cores, while TraCI multi-instance control is constrained by its single-controller, step-by-step execution model. Spatial partitioning produced more limited gains, mainly because communication overhead increased with the number of partitions, while METIS gave the strongest overall performance.

1 Introduction

In the Netherlands alone, 75,800 traffic related accidents occurred ("Traffic accidents fall for third consecutive year; Rotterdam and Amsterdam lead", 2026), which is approximately 200 accidents per day. Traffic congestion imposes large and growing costs on urban areas, such as accidents, lost time, and increased emissions. Rebuilding road infrastructure or changing traffic policy is expensive and mainly irreversible. Tools such as SUMO (López et al., 2018) track every vehicle individually using a car-following model, in which each vehicle continuously adjusts its speed in response to the vehicle ahead. These detailed simulations make the results realistic, but are also computationally expensive.

This computational cost motivates the use of High-Performance Computing techniques. In this report, we evaluate three approaches for reducing the runtime of SUMO-based traffic simulation. The first approach is embarrassingly parallel execution, where independent simulations are distributed across multiple cores. The second approach is TraCI-based multi-instance control, where several SUMO instances are advanced by one controller process. The third approach is MPI-based spatial partitioning, where the road network is divided into subregions and simulated by multiple communicating processes. Together, these approaches cover both common use cases for traffic simulation: running many independent scenarios and attempting to accelerate a single detailed simulation.

The expensive computation compounds due to the fact that most research needs thousands of simulations rather than a few. Assessing how robust a result is to random variation in demand requires repeated Monte Carlo runs.

1.1 State of the art and research gap

Spatial Partitioning for a car following simulation model has been investigated before. Acosta et al., 2016 showed how a SUMO simulation can be parallelized on theoretical grid network, whereas Chen et al., 2020 showed its usage in real life scenario of a highly used network of Doha Corniche. Both research papers established that domain decomposition can reduce the runtime of a single large simulation.

Two gaps remain in the current research. First, prior work has concentrated on congested networks. It is not obvious that these results transfer to the less saturated networks typical of mid-sized European cities,

where there is less computation per partition to amortize the communication overhead. We therefore study the road network of Maastricht, the Netherlands. Secondly, and more importantly, Chen et al., 2020 commit to a single graph partitioning algorithm (METIS) without comparing alternatives. The choice of partitioning algorithm determines the boundary structure, and therefore directly affects communication overhead, load balance, and accuracy. Our contribution is to fill that gap: we compare partitioning algorithms and measure how the choice propagates to speedup, load balancing, and accuracy.

1.2 Research Questions

Our main research question was:

> How can HPC techniques be used to speed up car following model traffic simulations in a low-traffic networks, and how well do they perform in terms of speed up, accuracy and load balancing?

To address this, we further explored the following subquestions grouped by the gap each addresses:

Characterizing the network.

- **RQ1.** Which congestion bottlenecks does the Maastricht network exhibit, and how robust are they to stochastic variation in traffic demand?

Accelerating many independent runs.

- **RQ2.** How well does embarrassingly parallel Monte Carlo execution scale with the number of cores, and what limits its speedup?
- **RQ3.** How does TraCI-based multi-instance control compare to embarrassingly parallel execution in terms of throughput?

Accelerating a single simulation via spatial partitioning.

- **RQ4.** How accurate is a spatially partitioned SUMO simulation on a low-traffic network such as Maastricht?
- **RQ5.** How well does spatially partitioned simulation scale with the number of cores used?
- **RQ6.** Which network partitioning strategy offers the best load balancing directly testing the single-algorithm assumption made by Chen et al. (2020)?

2 Methods

2.1 Baseline establishing and Monte Carlo

2.1.1 Baseline bottleneck detection

The baseline is a single deterministic SUMO López et al., 2018 run of the Maastricht network over 3600 s of simulated time. Per-edge aggregate measures are combined into a single scalar congestion score.

2.1.2 Monte Carlo robustness

In order to differentiate structural bottlenecks from artifacts of a demand scenario, the simulation is repeated as a Monte Carlo ensemble in which the network and route set are held fixed and only the demand is varied: per run we perturb vehicle departure times, the sampled vehicle count, and per-vehicle speed factors. The exact distributions, sampling procedure, and ensemble sizes are given in Section 4.2. Each run is scored with the congestion equation, and in each run the five highest-scoring edges are flagged. For each edge we record its top-5 frequency across the ensemble and report a Wilson 95% Wilson, 1927 confidence interval on that frequency. Differences in frequency between edges are tested with two-proportion z-tests.

2.2 Embarrassingly Parallel Monte Carlo

The Monte Carlo runs are embarrassingly parallel: each run is an independent SUMO sub-process with no communication with other processes, so the ensemble is distributed across multiple cores with one run per worker. Wall time is measured per configuration, and speedup and efficiency are computed relative to the single-core run. The strong-scaling curve is fitted to Amdahl's Law, $S(N) = 1/(s + (1 - s)/N)$; the serial fraction is reported and the speedup is used to interpret the observed plateau.

2.3 TraCI Multi-Instance Parallelism

We use TraCI to control N live SUMO instances simultaneously from a single Python orchestrator, advancing every instance through the simulation one step at a time. Because the orchestrator steps each instance and waits for its reply before advancing the next, the coordination layer is serial. We measure per-step latency and throughput as the instance count increases. The measurements therefore characterize TraCI as a control-granularity rather than a throughput-oriented strategy.

2.4 Spatial Partitioning

Spatial Partitioning is a parallelization technique, where the road network of a city is partitioned into a fixed number of partitions. Each partition simulates only the state of vehicles inside it, and communicates with its neighboring partitions about any incoming/outgoing vehicles. Our implementation of Spatial Partitioning is based on Chen et al., 2020. Used partition algorithms are described in Section 2.5.1.

2.4.1 Types of edges

Edges in each partition are divided into three groups:

- **outgoing edge** - goes from current partition to a neighbor. We have to communicate with neighbor about any new vehicle crossing this edge, and receive info about the last vehicle on each lane on this edge
- **incoming edge** - goes from neighbor to current partition. We have to receive information from neighbor about any new vehicle crossing this edge and send back information about last vehicle on each lane on this edge
- **inside edge** - goes from current partition to current partition, we don't have to communicate state of the vehicles on this edge to any partition.

2.4.2 Simulation loop

We used a custom MPI datatype for sending information about a vehicle. It contains the following fields - position, speed, acceleration, angle, edgeID, laneIndex. Standard simulation loop is modified to look as following:

1. Advance a step in SUMO simulation
2. Manage handoff of vehicles
   (a) Gather data about vehicles on outgoing edges
   (b) Send data about vehicles on outgoing edges to proper partitions
   (c) Receive data about vehicles on incoming edges
   (d) Apply changes based on received data
3. Manage ghost vehicles
   (a) Gather data about vehicles on incoming edges
   (b) Send data about vehicles on incoming edges to proper partitions
   (c) Receive data about vehicles on outgoing edges
   (d) Apply changes based on received data

2.4.3 Border edges management

Ghost vehicles are used to preserve boundary awareness between neighboring partitions. A partition only directly simulates vehicles that it owns, but vehicles close to the edge can still influence traffic behavior on the other side. To solve this, neighboring partitions exchange lightweight copies of boundary vehicles. These copies are inserted into the local SUMO simulation as ghost vehicles.

A ghost vehicle does not transfer ownership of the original vehicle. The original vehicle remains still in its own partition, while the neighboring partition just keeps an update copy of state. This allows edge lanes to react to nearby vehicles from adjacent partitions without duplicating the actual vehicle. Ghost vehicles are updated when new state information is received and removed if they are not refreshed for several simulation steps.

2.4.4 Assessment

We assess accuracy of parallel implementation by comparing the travel time of each vehicle as well as average speed and density and at each edge between baseline simulation and multi-core simulation using percentage relative error.

To assess speedup and efficiency we measure simulation time of both single core and multi core implementation and calculate needed values.

For load balancing we measure how time is distributed across MPI processes. For each process, we record the time spent on local simulation and the time spent on communication. Differences between processes indicate whether the workload is evenly distributed or whether some partitions become redundant.

2.5 Partitioning Algorithms

Chen et al., 2020 use METIS as the sole partition algorithm, to partition the Maastricht network the pa-

per uses five algorithms from three types of algorithms: graph-topological, geometric and community-based and measure how that choice propagates into speedup, load balancing, and accuracy. Each algorithm does the same job of assigning each of the networks junctions to one of k partitions, which then induces inside, incoming and outgoing edges from section 2.4.

2.5.1 Used algorithms

1. METIS (k-way) - METIS is a k-way partition algorithm of Karypis and Kumar, 1997, invoked though gpmetis. METIS minimizes the edge-cut subject to a balance constraint on vertex weights, and is the standard used by QarSUMO Chen et al., 2020. We supply the network as an unweighted graph, so the algorithm optimizes purely topologically rather than traffic-aware weights.
2. Spectral bisection - Recursive bisection uses the Fiedler vector which is an eigenvector of the eigenvalue of the graph. At each step the largest remaining group is split at the median of its Fiedler vector, which is repeated until k groups exist. The algorithm targets low edge-cut by exploiting global connectivity structure rather than local.Urschel and Zikatanov, 2014
3. Louvain community detection - This algorithm maximizes community detection Blondel et al., 2008. The network's natural communities are detected first, then greedily bin-packed into k balanced partitions. This respects dense traffic clusters, but does not directly control edge-cut once communities are merged.
4. KD coordinate bisection - A geometric recursive bisection where the largest group is repeatedly split at the median of its longer spatial axis, ignoring connectivity entirely.
5. Grid - The bounding box is tiled into a square grid of geographical cells. Each junction is assigned by its coordinate.

3 Implementation

We base our research on SUMO, which is used as the traffic simulation engine. Instead of implementing the vehicle movement ourselves, we used SUMO to advance the simulation state and to manage vehicle behavior. The parallelization is therefore built around it, rather than replacing it.

In the implementation, we interact with SUMO through the libsumo interface. The `Simulation` API is used to start, step, and close each local simulation instance. the `Vehicle` API is used to read or update vehicle state, including position, speed, acceleration, angle, route, and edge information. It is also used to add vehicles received from neighboring MPI ranks and remove vehicles whose ownership has been transferred. The `Lane` and `Edge` API's are used to inspect boundary lanes and identify vehicles located on incoming or outgoing edges. The `Route` API is used to create missing routes when vehicles or ghost vehicles are inserted into a local SUMO instance.

3.1 Bottleneck Detection

To evaluate the location and robustness of bottlenecks in the Maastricht network, we implemented a data-driven pipeline using a custom formula that combines car density, speed, and waiting time, to calculate clear traffic score for each street. This ensures that the identified bottlenecks are significant, not just random. The road network `maastricht_connected.net.xml` was produced from an OpenStreetMap export with `netconvert`, using the flag `--keep-edges.components 1` to discard disconnected fragments and retain a single connected component. Per-edge aggregate measures are written by SUMO to `edgedata.xml`. First, a single baseline loop is executed to record microscopic travel metrics (which streets are blocked, and which are not) across the network. Our implementation script automatically applies the multi-variable min-max normalization to calculate it.

To calculate the stochastic robustness of the found bottlenecks, the framework repeatedly executes varying demand scenarios over planned, randomized Monte Carlo tests. Our pipeline tracks the top five edges that are the most congested over all the runs to check if the data stabilizes, by counting how often each appears in the top five. Finally, the script saves the final frequencies into a dataset, to calculate a 95% Wilson confidence interval.

3.2 Research Question 2

To evaluate the multi-core scalability and speedup limits of our embarrassingly parallel Monte Carlo execution, we implement an automated hardware benchmarking framework. To bypass the Python's Global Interpreter Lock (GIL), we used ProcessPoolExecutor class from the concurrent.futures library. That allows us to run up to 64 independent SUMO subprocesses simultaneously across available CPU cores.

To examine execution efficiency, the framework takes the execution loop wall-time monitors, that automatically log the total duration across core counts (from 1 up to 64). The resulting performance metrics are then exported to a structured dataset to evaluate the scaling efficiency and find the exact limit where adding more cores stops speeding up the simulation (strong-scaling acceleration measured by Amdahl's law formula).

3.3 Spatial Partitioning

To scale the traffic simulation horizontally, we divided the network into disjoint geographic partitions, where each MPI process is assigned to and works on a different CPU core. Instead of using the standard network-based TraCI interface, each process uses `libsumo` to interact directly with its own local simulation through in-memory function calls, drastically reducing API overhead. Inter-process synchronization and over-border communication are handled using OpenMPI.

To minimize the communication latency, we implemented an MPI datatype (`VehicleInfoMPI`), that transfers only vital kinematic information. Network boundaries use non-blocking `MPI_Isend` and `MPI_Irecv` operations. At every 0.05s timestep, partition updates its information, and if necessary, sends ghost vehicles data to neighboring zone.

At the end of the simulation, each partition's sub-simulation outputs its local data files - tripinfo.xml and edgeinfo.xml. They are then merged into one global dataset for further analysis.

3.4 Partitioning Network

As mentioned in Methods, we use METIS(Karypis and Kumar, 1997) to create baseline partitioning. We also use following python packages

- sumolib - to convert sumo format graph into a METIS format graph
- lxml - to traverse core xml files

Vehicles are assigned to the partition of their starting edge. Each partition, however, contains all the existing routes, so that communication is more efficient.

3.4.1 Communicating with SUMO

We use libsumo, a C++ api for SUMO, to communicate with a running simulation. Our spatial partitioning framework uses a high-performance direct memory backend. Rather than using the network-driven TraCI interface for running the domain decomposition loop, the parallel program is compiled with the C++ libsumo library. That way each process core can call simulation functions directly inside the computer's local memory, bypassing the delay. Since the network is divided into separate parts, each core operates on an isolated SUMO instance that generates its own local logs. At the end of the run, our pipeline merges all the separate files into one single global dataset.

3.4.2 MPI communication

The inter-process communication between the CPU cores is done using OpenMPI (Gabriel et al., 2004), using a structured batch protocol. To safely send vehicle information across the boundaries, we created a VehicleInfoMpi 8.1 data package that includes all of the car's essential details, such as its coordinates, speed, angle, edge ID, acceleration and lane index, into a single chunk of data.

To prevent the whole simulation from a deadlock while transferring the data, our communication protocol is based entirely on non-blocking operations. During the handoff phase (when a car goes from one divided section to another), a core packs the car's detailed information into an array, and sends them over the border using `MPI_Isend` before deleting it locally. At the same time, the neighboring zones (sections) are waiting for it with `MPI_Irecv` to catch the matching incoming car. An `MPI_Waitall` pauses the core to ensure that every data package that is

being transferred is safely caught before the next simulation step starts.

3.4.3 Ghost Vehicles

In the implementation, ghost vehicles are handled differently from normal vehicles. Each process first identifies vehicles on boundary edges that are relevant to neighboring partitions. For ghost updates, vehicles located on incoming boundary edges are collected and their state is stored into the custom `VehicleInfoMpi` datatype. 8.1. Ghost vehicle data are exchanged between MPI ranks using the same batch communication mechanism as normal vehicle transfer.

In SUMO, ghost vehicles are created with a modified ID by adding a $g_$ prefix to the original vehicle ID. If the required route does not yet exist in the local SUMO instance, it is created before adding the ghost vehicle. The ghost is then placed at the received coordinates using *moveToXY*, and its speed is updated to match the sender's state.

To avoid keeping outdated copies, each process periodically checks the last update step of every stored ghost vehicle. If a ghost has not been refreshed within the configured timeout, it is removed both from SUMO and from the local ghost maps. This ensures that ghost vehicles represent only recently observed boundary vehicles.

3.5 Measuring load balancing

To evaluate how well the workload is balanced across different parallel processes, our framework separates computational work from communication overhead directly inside the simulation loop. We used the `MPI_Wtime()` to precisely track the two different time metrics for every process rank - simulation time and communication time.

The first one, simulation time, measures how long a CPU core spends only on calculating car physics and traffic movements. The second one, communication time, measures how long a CPU core spends on packing, sending and receiving the data from other cores through MPI.

After the process is finished, rank 0 calls the `MPI_Gather()` function, collects all the information, and structures it into a clear JSON file. At the end of the run it generates a results.json file that we use to analyze where exactly the bottlenecks and sitting idle occurs.

To quantify load balance we look at the wall clock runtime. At runtime, balance is measured from the simulation itself; for every MPI rank we record total simulation time and total communication time with `MPI_Wtime()`, and define the relative compute imbalance of a run as:

$$(t_{max} - t_{min})/t_{max} \tag{1}$$

over the ranks, where 0 denotes a perfectly balanced run. We also report the mean communication fraction:

$$t_{comm}/(t_{sim} + t_{comm}) \tag{2}$$

per rank, which isolates overhead from useful work.

4 Experiments

4.1 Baseline creation

The formula for the scoring of the edges:

$$s_e = 0.3\,\hat{\rho}_e + 0.2\,\hat{o}_e + 0.3\,(1 - \hat{v}_e) + 0.2\,\hat{w}_e, \tag{3}$$

where $\hat{\rho}_e$, $\hat{o}_e$, $\hat{v}_e$ and $\hat{w}_e$ are the density, occupancy, mean speed and waiting time of edge e, each min–max normalized to $[0, 1]$ against the maximum observed for that measure within the run, so that $s_e = 1$ denotes complete gridlock. Density and slowness $(1-\hat{v}_e)$ are the most direct kinematic indicators of flow breakdown and carry the dominant weight (0.3), while occupancy and waiting time are retained as correlated secondary confirmation (0.2 each). Edges are ranked by s_e, and the *congested* threshold τ is set to the score of the fifth-ranked edge ($\tau = 0.641$ in the baseline run), giving a data-driven cut-off rather than an arbitrary constant.

4.2 Monte Carlo

The embarrassingly parallel Monte Carlo experiments were run on one node of the DACS university cluster, sweeping core counts of $\{1, 2, 4, 8, 16, 32, 64\}$ to measure strong scaling. Each core count was repeated 3 times and wall time recorded per run; the single-core figure was measured locally as the reference for speedup and efficiency. While keeping

the network constant, each run takes an independent demand realization from a fixed baseline route set (`routes.rou.xml`). One integer seed determines randomization in its entirety: `generate_demand.py` generates run k with seed k, initializing a dedicated `random.Random(k)` stream, making it possible to replicate the complete ensemble using just the run indices. The generator (i) samples a number of vehicles from a Gaussian $\mathcal{N}(18000, 3600)$, truncated to $[500, N_{\text{base}}]$, where N_{base} is the number of vehicles in the baseline; (ii) draws that many vehicles *without replacement* from the baseline; (iii) perturbs each vehicle's departure time by additive Gaussian jitter $\mathcal{N}(0, 180\,\text{s})$, clamped to $[0, 3598]$ s; and (iv) assigns each vehicle a speed factor $\mathcal{N}(1.005, 0.099)$, clamped to $[0.75, 1.25]$. Vehicles are then re-sorted by departure time and re-indexed, as required by SUMO.

4.3 TraCI Multi-Instance

The orchestrator was run at 8, 16, and 32 concurrent SUMO instances. Each instance was launched as a subprocess listening on its own TraCI TCP port ($8813 + i$), and the orchestrator connected to all of them before stepping. All instance counts were run on a single node of the DACS university cluster using 32 cores.

4.4 Spatial Partitioning

4.4.1 Hardware and Software Stack

Experiments were run on DACS university cluster. Only one node was used during every experiment to avoid unnecessary communication overhead. For each simulation we set a `step-length` of 0.05.

4.4.2 Experiments configuration

We tested 5 demand models generated using SUMO-native tools, run them on 2, 4, 8, 16 and 32 cores. Each triple of (demand model, partition algorithm, number of cores) was run 3 times. Baseline one-core models where run in total 15 times (5 demand models, 3 runs each). This sums up to 390 runs in total.

4.4.3 Measuring accuracy

During each run information about vehicles is logged into `tripinfo.xml`, while information about edges is logged into `edgeinfo.xml`. Accuracy w.r.t. vehicles is assessed based on travel time, accuracy w.r.t. edges is based on average edge speed and edge density. For each vehicle and edge from a run we then calculate a relative error with formula

$$\left|\frac{x - \hat{x}}{x}\right|$$

where x is a baseline value, and $\hat{x}$ is a parallelization value. Results are then accumulated based on tuple (partition algorithm, number of cores) and average values and standard deviations are accessed.

4.4.4 Measuring speedup and efficiency

To calculate speed up we use standard equation

$$S = \frac{t_s}{t_p}$$

where t_s is a serial runtime and t_p is a parallel runtime. Efficiency is also based on standard equation

$$E = \frac{S}{n}$$

where S is a speed up and n is a number of cores used.

4.4.5 Measuring load balancing

Within every MPI rank two quantities are gathered: total simulation time and total communication time. At the end of the run rank 0 gathers both values to report as a json file.

From these values we derive the relative compute imbalance and the mean communication fraction to capture parallel overhead. Both are averaged over the 15 samples per cell and reported with their standard deviations.

5 Results

5.1 Baseline construction, Monte Carlo experiments, and TraCI

Table 1 reports the five highest-scoring edges of the deterministic baseline run, ranked by congestion score (Eq. 3). Four of the five belong to a single corridor (road `7127031`, appearing in both directions and both segments), with one junction edge (`47307823#1`) in second place.

5.1.1 Monte Carlo Table 2 reports, for each edge appearing in any run's top-5, the fraction of the 100 Monte Carlo runs in which it was flagged, a Wilson 95% confidence interval on that frequency, and

Table 1
Top-5 congested edges, baseline run.

Rank	Edge ID	Score	Geographic cluster
1	-7127031#1	highest	Cluster A — main corridor
2	47307823#1	—	Cluster B — junction
3	-7127031#0	—	Cluster A — main corridor
4	7127031#1	—	Cluster A
5	7127031#0	0.641 (thr.)	Cluster A

Table 2
Top-5 frequency across 100 MC runs, with Wilson 95% CIs. Verdict: *Robust* = structural (in top-5 in ≥75% of runs); *Demand* = demand-sensitive (peak only); *Revealed* = not in the RQ1 top-5 but surfaced by Monte Carlo.

Edge ID	Freq.	95% CI	Score	Verdict
-7127031#1	78%	[68.9, 85.0]	0.698	Robust
47307823#1	77%	[67.8, 84.2]	0.736	Robust
-7127031#0	75%	[65.7, 82.5]	0.707	Robust
7127031#1	46%	[36.6, 55.7]	0.654	Demand
7127031#0	36%	[27.3, 45.8]	0.638	Demand
38785608#0	35%	[26.4, 44.7]	—	Revealed
636917939#0	31%	[22.8, 40.6]	—	Revealed
651761171#0	25%	[17.5, 34.3]	—	Revealed

its mean score when present. The three highest-frequency edges (75–78%) agree with the baseline corridor and junction; their confidence intervals overlap, indicating they are statistically indistinguishable, a clear gap separates them from the fourth edge (46%). Three edges absent from the baseline top-5 (`38785608#0`, `636917939#0`, `651761171#0`) appear in 25–35% of runs.

Table 3
Strong-scaling results, embarrassingly parallel MC.

Cores	Wall time	Speedup	Efficiency
1	173.1 s (local ref)	1.00×	100%
2	93.7 s	1.85×	92%
4	52.5 s	3.30×	82%
8	28.9 s	6.00×	75%
16	—	13.3×	83%
32	—	14.9×	47%
64	—	13.2×	21%

Table 3 reports wall time, speedup, and parallel efficiency for the embarrassingly parallel Monte Carlo batch across 1–64 cores. Speedup rises to 14.9× at 32 cores before declining to 13.2× at 64, while efficiency falls from 92%.

Table 4
TraCI multi-instance scaling.

Instances	Mean step latency	Throughput (sims/min)
8	~6.6 ms	~20
16	~13 ms	~18
32	~32.5 ms	~16.4

5.2 Spatial Partitioning

5.2.1 Accuracy

Tables 8, 9, 10 show mean relative error (±standard deviation σ) per core count and algorithm used. For every algorithm relative error increases with number of cores used. Lower values indicate better accuracy. There is a visible correlation between number of cores used and value of relative error for every algorithm. GRID performs best in trip length accuracy, having one of the lowest error values across all core counts. GRID and METIS perform similarly in terms of edge speed error, both have however significant values of σ. Looking at relative error of edge density, METIS shows the best accuracy. Across all metrics METIS appears to be the most balanced.

5.2.2 Speed up and Efficiency

Table 6 shows average speed ups and efficiencies for each algorithm depending on number of cores. Same information is also presented as a graph in Figure 1. At 2 cores, every algorithm runs slower than the serial baseline. Speedup then rises with core count and peaks at 16 cores for all algorithms except KD, reaching 2.68x for METIS configuration, 2.39x for Louvain, 2.24x for KD, 2.15x for Spectral and 1.66x for Grid. At 32 cores most algorithms regress, while KD continues to improve to 2.49x. Efficiency decreases sharply with core counts for each algorithm. Run variance is small for each algorithm except for Louvain at 32 cores, which would indicate one or more outlier runs.

Table 7 shows the relative imbalance and communication fraction. Compute imbalance grows sharply with core count for all algorithms from 0.02-0.13 at 2 cores to 0.95-0.99 at 32 cores. At low core counts the geometric and community methods ar best balances, from 8

cores onward the METIS configuration show the lowest imbalance, while grid is the most imbalanced. The communication fraction rises from 1-2% time at 2 cores to 37-49% at 32 cores.

6 Discussion

6.1 Baseline construction

The fact that a single corridor occupies 4 out of the top-5 spots is the key finding: congestion in this scenario is not diffuse but concentrated at one structural pinch point. Because the corridor saturates in both directions, the cause is more consistent with a fixed capacity rather than a peak-demand surge. This type of claim cannot be justified with a single deterministic run, therefore justifying the second research question.

6.1.1 Robustness of the baseline

The statistics justify the claim. There is genuine and significant discontinuity between the top three edges and the rest. The corridor and the junction identified in RQ1 are therefore structural bottlenecks, while the remaining corridor segments fail only under heavier loads and are therefore demand-sensitive.

6.1.2 Speedup and efficiency

In speedup terms, Amdahl's law Amdahl, 1967 predicts a linear approach to $1/s$, however the observed speedup falls from 14.9x on 32 cores to 13.2x on 64 cores. This reflects hyper-threading contention on the workload which is CPU-bound therefore additional parallelism doesn't yield better results. A contention aware model like Gunther's Universal Scalability law Gunther, 2007, would describe the full sweep more honestly than Amdahl alone, therefore fitting this is a natural extension. Secondly, the efficiency figures are also not monotonic. Under Amdahl's law, efficiency must decrease monotonically, however this is not observed on our results. This is due to different environments at running time. This is a caveat on our results which could not be fixed due to timing constraints.

6.1.3 Methodological and data limitations

Firstly, the congestion score is a weighted heuristic: the component weights and each component is normalized against the per-run maximum, so scores are comparable within a run but not in absolute terms across runs, only rankings and threshold crossings are. Secondly, the demand models are synthetic: departure times, vehicle counts and speed factors are drawn from Gaussian distributions rather than fitted to observed Maastricht traffic counts due to the lack of data provided by the municipality. The network itself is an OSM export, inheriting any geometric or connectivity inaccuracies of that source. Thirdly, the scaling measurements have a hardware caveat: the single-core reference time was recorded locally while the higher core-count figures were measured on the cluster.

6.2 Spatial Partitioning

6.2.1 Speedup

From the results we can conclude that speedup is communication-bound, not compute-bound. The speedup at 2 cores and the efficiency decay show that the dominant cost of our design is inter-partition synchronization, not the partition quality. This is confirmed by the communication fraction, which climbs from 1% to 40-49% of per rank time as cores increase, each added partition creates more border edges, and the per step barrier means a rank can only advance as fast as the slowest neighbor. The 16 core peak followed by regression at 32 cores is consistent with this: beyond 16 partitions, the growth in border edge communication and load imbalance outweighs the reduced partition computation time.

6.2.2 Accuracy

Vehicle accuracy did not give us great results. As expected, the vehicle travel time error (27%-82%) increases with the number of partitions. While we did not record exact values, during parallel simulations, we notice much more car collisions and emergency braking inside each simulation compared to the baseline. This is likely caused by ghost vehicles being inserted via `moveToXY`, which teleports them to their updated position rather than moving smoothly, causing unrealistic braking reactions in nearby vehicles. Comparing to QarSUMO (5%), our accuracy is much worse. We cannot say for sure whether the poor accuracy is caused by the use of low-traffic network with more vehicles moving at higher speeds over the border edges, or by a wrong implementation.
Edge accuracy is better than vehicle accuracy, achieving < 0.2 in edge speed and < 0.3 in edge density for METIS. These values are still too large for parallel sim-

ulation to be used in detecting bottlenecks using established Equation 3.

6.2.3 Load balancing

From the results we can see that at 2 cores the connectivity- and community-aware methods produce the most even partitions: Spectral (0.02) and Louvain (0.05) achieve the lowest compute imbalance, because at this coarse granularity the network splits cleanly along its main structural axis. As the number of partitions grows, however, only METIS consistently keeps both compute imbalance and communication fraction low, owing to its edge-cut minimization constraint. Grid is consistently the worst balanced, because tiling the network into geometric boxes ignores where traffic concentrates and tends to cut repeatedly through dense sections of the network.

6.2.4 Implementation and scaling limitations

One of the main limitation, as mentioned in Section 6.2.2, is the correctness of our implementation. It could be potentially improved by decreasing a `step-length` in simulations, resulting in smoother vehicle movement and more continuous exchange of information between neighboring simulations. This would however cause wall clock runtime to be longer. Another limitation is the fact that we didn't partition the network based on usage, but only on topology. As seen in Chen et al., 2020, demand-driven partitioning can result in better simulations. Unlike Chen et al., 2020 we also haven't employed any approximations for movement of vehicles in a traffic jam - this is probably also one of the causes of worse strong scaling.

Our findings are also limited by the number of experiments run. To address this we would have to either include better statistical analysis or increase the number of experiments.

7 Conclusion

7.1 Baseline identification

The baseline analysis identified congestion in the Maastricht network on two structures rather than diffused across the network. Because the structures saturates bidirectionally, the cause is more consistent with a fixed capacity constraint on the link than with a one–directional peak surge. Which motivated RQ2

7.2 Monte Carlo simulations

The practical conclusion is that bottleneck identification in this network requires an ensemble, not a single simulation, to separate structural failures from demand artifacts and to expose hidden ones.

7.3 Spatial Partitioning

Parallelization of a car following model simulation through spatial partitioning of a low-traffic network has proven to be quite limited in accuracy, speed up and load balancing. This research is consistent with QarSUMO's choice for the METIS algorithm Chen et al., 2020 which chose METIS as the sole partitioning algorithm for the network. We found the speedup to be lower for Maastricht than Corniche - max 2.68 speedup for Maastricht vs 5.70 for Corniche. The largest difference can be seen in relative errors of travel times - max 0.76 for Maastircht using METIS(Karypis and Kumar, 1997) vs 0.06 for Corniche. Further work should validate the correctness of our implementation to confirm these observations.

References

Acosta, A., Espinosa, J., & Espinosa, J. (2016). Distributed simulation in sumo revisited: Strategies for network partitioning and border edges management. *Proceedings of the 4th SUMO User Conference*, 61–71.

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*, 483–485. https://doi.org/10.1145/1465482.1465560

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, *2008*(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/p10008

Chen, H., Yang, K., Rizzo, S. G., Vantini, G., Taylor, P., Ma, X., & Chawla, S. (2020). Qarsumo: A parallel, congestion-optimized traffic simulator. *Proceedings of the 28th International Conference on Advances in Geographic Information Systems*, 578–588. https://doi.org/10.1145/3397536.3422274

Gabriel, E., Fagg, G. E., Bosilca, G., Angskun, T., Dongarra, J. J., Squyres, J. M., Sahay, V., Kambadur, P., Barrett, B., Lumsdaine, A., Castain, R. H., Daniel, D. J., Graham, R. L., & Woodall, T. S. (2004). Open MPI: Goals, concept, and design of a next generation MPI implementation. *Proceedings, 11th European PVM/MPI Users' Group Meeting*, 97–104.

Gunther, N. J. (2007). *Guerrilla capacity planning: A tactical approach to planning for highly scalable applications and services*. Springer. https://doi.org/10.1007/978-3-540-31010-5

Karypis, G., & Kumar, V. (1997). Metis: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices.

López, P. Á., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wießner, E. (2018). Microscopic traffic simulation using SUMO. *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2575–2582. https://doi.org/10.1109/ITSC.2018.8569938

Traffic accidents fall for third consecutive year; Rotterdam and Amsterdam lead [Reporting by ANP and NL Times. Accessed: 2026-06-09]. (2026). *NL Times*. https://nltimes.nl/2026/01/17/traffic-accidents-fall-third-consecutive-year-rotterdam-amsterdam-lead

Urschel, J. C., & Zikatanov, L. T. (2014). Spectral bisection of graphs and connectedness. *Linear Algebra and its Applications*, *449*, 1–16. https://doi.org/https://doi.org/10.1016/j.laa.2014.02.007

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, *22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

8 Appendix

8.1 VehicleInfoMpi Structure

The following structure was used to store and exchange vehicle state between MPI processes:

```
struct VehicleInfoMpi {
    double vehX;
    double vehY;
    double vehSpeed;
    double angle;
    double acceleration;
    int laneIndex;
    char vehicleID[20];
    char routeID[20];
    char edgeID[20];
};
```

8.2 Spatial Partitioning - Graphs and Tables

Table 5
Mean wall-clock runtime in seconds per partition algorithm and core count, reported as mean ± standard deviation over 15 runs.

Algorithm	2	4	8	16	32
GRID	48.62 ± 1.59	32.71 ± 1.01	37.10 ± 1.46	26.60 ± 0.90	21.19 ± 1.16
KD	47.44 ± 0.92	30.23 ± 1.23	23.60 ± 1.08	19.78 ± 1.02	17.77 ± 0.63
LOUVAIN	48.77 ± 4.95	33.72 ± 1.77	24.83 ± 0.67	18.53 ± 1.11	31.54 ± 34.88
METIS	46.13 ± 1.78	28.77 ± 1.06	20.09 ± 1.10	**16.50 ± 0.68**	17.50 ± 0.74
SPECTRAL	46.25 ± 1.17	30.38 ± 0.96	24.71 ± 1.51	20.66 ± 1.31	18.18 ± 0.89

Table 6
Speedup and efficiency per partition algorithm and core count. Speedup is measured against the serial baseline, and efficiency is computed as speedup divided by core count.

Algorithm	2	4	8	16	32
GRID	0.91 / 46%	1.35 / 34%	1.19 / 15%	1.66 / 10%	2.09 / 7%
KD	0.93 / 47%	1.46 / 37%	1.88 / 23%	2.24 / 14%	2.49 / 8%
LOUVAIN	0.92 / 46%	1.31 / 33%	1.78 / 22%	2.39 / 15%	2.07 / 7%
METIS	0.96 / 48%	1.54 / 38%	2.21 / 28%	2.68 / 17%	2.53 / 8%
SPECTRAL	0.96 / 48%	1.46 / 36%	1.80 / 22%	2.15 / 13%	2.44 / 8%

Table 7
Load balancing per partition algorithm and core count. Load balance is computed as $(t_{\max} - t_{\min})/t_{\max}$ over ranks, where 0 indicates perfect balance. The communication fraction is computed as $t_{\text{comm}}/(t_{\text{sim}} + t_{\text{comm}})$ and averaged over ranks.

Algorithm	2	4	8	16	32
GRID	0.09 / 1%	0.49 / 5%	0.98 / 24%	0.99 / 39%	0.98 / 47%
KD	0.13 / 2%	0.41 / 5%	0.72 / 14%	0.92 / 26%	0.95 / 41%
LOUVAIN	0.05 / 2%	0.45 / 9%	0.74 / 18%	0.90 / 24%	0.97 / 49%
METIS	0.07 / 1%	0.47 / 4%	0.61 / 10%	0.92 / 22%	0.95 / 37%
SPECTRAL	0.02 / 5%	0.37 / 5%	0.78 / 13%	0.88 / 27%	0.95 / 42%

Table 8
Accuracy of the parallel runs: mean relative error of per-trip duration versus the serial baseline, per partition algorithm and core count.

Algorithm	2	4	8	16	32
GRID	0.28 ± 0.019	0.48 ± 0.008	0.60 ± 0.005	0.66 ± 0.003	0.71 ± 0.004
KD	0.34 ± 0.015	0.47 ± 0.009	0.62 ± 0.005	0.73 ± 0.002	0.82 ± 0.002
LOUVAIN	0.56 ± 0.004	0.68 ± 0.003	0.71 ± 0.004	0.75 ± 0.003	0.75 ± 0.004
METIS	0.27 ± 0.019	0.46 ± 0.007	0.63 ± 0.005	0.69 ± 0.003	0.76 ± 0.002
SPECTRAL	0.31 ± 0.016	0.52 ± 0.012	0.62 ± 0.018	0.76 ± 0.019	0.80 ± 0.019

Table 9
Accuracy of the parallel runs: mean relative error of edge average speed versus the serial baseline, per partition algorithm and core count.

Algorithm	2	4	8	16	32
GRID	0.088 ± 0.067	0.072 ± 0.027	0.120 ± 0.034	0.143 ± 0.040	0.184 ± 0.026
KD	0.222 ± 0.134	0.197 ± 0.071	0.270 ± 0.093	0.317 ± 0.131	0.329 ± 0.041
LOUVAIN	0.108 ± 0.049	0.104 ± 0.029	0.160 ± 0.054	0.145 ± 0.047	0.163 ± 0.053
METIS	0.066 ± 0.048	0.092 ± 0.048	0.202 ± 0.135	0.178 ± 0.070	0.172 ± 0.047
SPECTRAL	0.077 ± 0.053	0.143 ± 0.058	0.157 ± 0.054	0.214 ± 0.067	0.364 ± 0.085

Table 10
Accuracy of the parallel runs: mean relative error of edge density versus the serial baseline, per partition algorithm and core count.

Algorithm	2	4	8	16	32
GRID	0.332 ± 0.337	0.130 ± 0.019	0.552 ± 0.509	0.403 ± 0.175	1.119 ± 0.800
KD	0.530 ± 0.456	0.407 ± 0.310	0.564 ± 0.395	0.838 ± 0.451	0.372 ± 0.024
LOUVAIN	0.362 ± 0.370	0.480 ± 0.505	0.260 ± 0.056	0.325 ± 0.148	0.567 ± 0.439
METIS	0.176 ± 0.043	0.202 ± 0.033	0.245 ± 0.084	0.255 ± 0.068	0.253 ± 0.021
SPECTRAL	0.143 ± 0.127	0.461 ± 0.316	0.519 ± 0.243	0.866 ± 0.541	0.458 ± 0.053

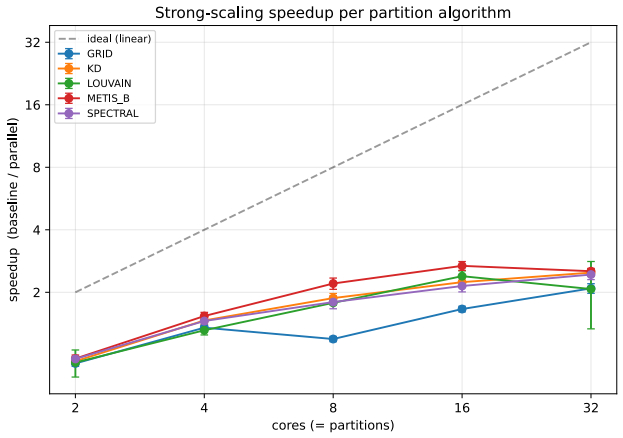


Figure 1. Strong-scaling speedup per partition algorithm